

Tabla de contenido

Análisis del Desafío 1 — Sweet Crush con bits en C++/Qt

Este documento es material de **estudio y planificación** para que ustedes construyan su propio informe con sus propias palabras. El profesor exige explícitamente que el informe no sea generado por IA, así que todo lo de aquí debe pasar por su cabeza y su teclado antes de ir al entregable oficial. Úsenlo para entender, discutir y decidir — no para copiar.

1. Qué están pidiendo, en palabras simples

Van a construir el motor de un juego tipo Candy Crush:

1. Hay un tablero rectangular de F filas $\times$ C columnas, lleno de fichas (6 tipos).
2. El jugador borra una ficha en una posición (fila, columna).
3. Las fichas de arriba “caen” para llenar el hueco (gravedad).
4. Arriba se generan fichas nuevas al azar para volver a llenar el tablero.
5. Si en algún momento quedan 3 o más fichas iguales seguidas (en fila o columna), se eliminan solas. Eso puede volver a generar huecos $\rightarrow$ más caídas $\rightarrow$ más combinaciones. A esa cadena se le llama **cascada**, y se repite hasta que el tablero quede estable.
6. Además, el tablero se puede **agrandar o achicar**: agregar/quitar una fila o columna, incluso en el medio del tablero (no solo en los bordes).

La parte “rara” y realmente evaluada (65% de la nota entre bits + memoria) es **cómo se guarda el tablero**: no como una matriz normal, sino como una tira continua de bits, sin desperdiciar ni un solo bit de relleno.

2. La duda de los bits, resuelta

No es ASCII. No es un código que “ya existe”. Ustedes lo inventan.

- Tienen 6 tipos de fichas. Con 3 bits hay 8 combinaciones posibles (000 a 111).
- Ustedes deciden libremente qué combinación es cada ficha. Por ejemplo:

Código	Significado (decisión propia)
--------	-------------------------------

000	Ficha A
001	Ficha B
010	Ficha C
011	Ficha D

Código	Significado (decisión propia)
100	Ficha E
101	Ficha F
110	Estado “vacío” (libre)
111	Estado “especial” (reservado, opcional)

Lo único obligatorio es: **6 combinaciones para las 6 fichas, y usar exactamente 3 bits por posición**. No pueden usar un int, un char, ni ningún tipo completo para guardar una sola ficha — cada ficha vive **solo** en 3 bits dentro de una tira compartida.

La parte que probablemente no habías entendido: el empaquetado sin relleno

Esto es la clave técnica de todo el desafío. El tablero **no** es una fila de bytes donde cada byte guarda una ficha con bits sobrantes desperdiciados. Es una tira continua:

ficha0(3 bits) + ficha1(3 bits) + ficha2(3 bits) + ficha3(3 bits) + ...

pegados sin ningún espacio entre ellos, y esa tira larga se corta en pedazos de 8 bits (bytes) porque así es como la memoria del computador está organizada. Como 3 no divide a 8, **una ficha puede terminar partida entre dos bytes**: parte de sus bits en el byte N y el resto en el byte N+1.

Ejemplo (tomado del enunciado, Figura 3): si una ficha necesita los bits en las posiciones 1 y 0 del byte N, y también el bit 7 del byte N+1, ustedes tienen que: - Leer esos 2 bits del byte N. - Leer ese 1 bit del byte N+1. - Combinarlos en el orden correcto para reconstruir el valor de 3 bits de la ficha.

Y viceversa para escribir: si quieren guardar la ficha 101 ahí, tienen que modificar **solo esos bits específicos** en cada byte, sin tocar los bits vecinos que pertenecen a otras fichas.

Por qué se necesitan los operadores de bits

No hay forma de hacer esto con operaciones normales (=, +, -) porque esas trabajan sobre el byte completo. Necesitan cirugía bit a bit:

- **& (AND)** — para “apagar” o **leer** bits específicos sin tocar los demás. Sirve como máscara: byte & 0b00000111 te deja solo los 3 bits bajos, apaga el resto.
- **| (OR)** — para **prender** bits específicos sin apagar los demás. byte | 0b00000101 prende esos bits, deja el resto igual.
- **^ (XOR)** — útil para **invertir** bits puntuales o para limpiar-y-poner en un solo paso combinado con AND.
- **~ (NOT)** — invierte todos los bits; se usa para construir máscaras “inversas” (ej: todo en 1 menos donde quiero escribir).
- **<< y >> (desplazamientos)** — para **mover** un valor a la posición exacta del byte donde debe quedar, o para traer bits desde una posición hacia el lugar donde los vas a leer.

La combinación típica para escribir un valor de 3 bits en una posición específica es: **apagar esos bits con AND+NOT (máscara), luego prenderlos con OR** del valor ya desplazado (<<) a su lugar. Para leer: **desplazar (>>) hasta que el bit que te interesa quede al final, y luego aplicar AND** con 0b111 para quedarte solo con esos 3 bits.

No necesitan memorizar fórmulas ahora — lo importante es entender el concepto: **AND/OR/NOT sirven para “editar bits sin dañar los vecinos”, y los desplazamientos sirven para “mover el valor a la posición correcta”**. El código concreto viene después, cuando ya tengan clara la lógica en papel.

Ejemplo numérico completo (tomado del enunciado)

Para la posición (fila=2, columna=3) en un tablero de 7 columnas:

1. **Índice lógico** = fila × columnas + columna = $2 \times 7 + 3 = 17$ (es la ficha #17 contando desde 0, en orden de lectura fila por fila).
2. **Bit inicial** = índice × 3 = $17 \times 3 = 51$ (el bit número 51 de la tira larga es donde arranca esta ficha).
3. **Byte correspondiente** = bit inicial ÷ 8 = $51 \div 8 = \text{byte } 6$ (con resto 3, es decir, arranca en el bit 3 dentro de ese byte).
4. A partir de ahí, dependiendo de si los 3 bits caben completos en ese byte o se reparten con el siguiente, se hace la lectura/escritura con las operaciones bitwise correspondientes.

Este cálculo (fila, columna) → índice → bit inicial → byte + posición dentro del byte es **la función más importante de todo el proyecto**. Todo lo demás depende de que esta cuenta esté bien.

3. La restricción que quizás se les pasó: uso de memoria al eliminar

El enunciado dice algo puntual y fácil de perder de vista:

“En el caso de eliminar filas o columnas, la modificación de la memoria física sólo debe realizarse cuando la utilización de la memoria caiga debajo del 65% respecto al tamaño actual del tablero.”

Es decir: si eliminan una fila, **no tienen que reducir la memoria físicamente de inmediato**. Solo deben reservar/liberar memoria real cuando el tablero lógico esté usando menos del 65% de lo que hay reservado. Esto evita hacer un realloc costoso cada vez que se borra una sola fila.

También dice:

“Las tramas de bits válidos siempre deben iniciar en el bit menos significativo, y si existiesen bits ‘inválidos’ o ‘vacíos’, estos deben agruparse a la izquierda de toda la trama.”

O sea: los datos “reales” van pegados desde el principio de la tira (bit 0 en adelante), y si sobra espacio reservado sin usar, ese sobrante queda al final (a la izquierda en la representación binaria, que es donde están los bits más significativos). Esto es importante para el diseño del “Arquitecto de Memoria”.

Esto implica que van a necesitar llevar registro de **dos cosas separadas**: cuántos bytes están *reservados* (memoria física) y cuántos bits están *realmente en uso* (tamaño lógico actual del tablero).

4. Lo que NO se puede usar (restricciones duras)

Esto condiciona directamente el diseño:

- ❌ struct, class, template
- ❌ objetos definidos por el estudiante
- ❌ `std::string` ni ningún tipo `string`
- ❌ STL (vector, list, map, etc.) ni estructuras dinámicas de librerías externas
- ❌ sintaxis de ANSI C (o sea, tiene que ser C++ real, no “C con //comentarios”)
- ✅ Sí: punteros, arreglos, memoria dinámica, funciones, framework Qt

Consecuencia práctica: todo el “estado del juego” (dimensiones, puntaje, cascadas, etc.) va a vivir en **variables globales** o pasarse por parámetros/punteros entre funciones — no en un struct que agrupe todo bonito. Por eso mencionabas “declaraciones globales” en tu idea; va en esa línea, aunque conviene limitar cuántas variables globales usan y documentar bien cada una.

5. Lógica macro: los módulos y sus “funciones reina”

Pensando de lo macro a lo micro, así se puede dividir el sistema en módulos (archivos .h/.cpp):

Módulo 1 — memoria (el traductor + el arquitecto)

Responsable de todo lo que toca la tira de bits directamente.

- `calcularBitInicial(fila, columna, numColumnas)` → devuelve en qué bit arranca esa ficha.
- `leerFicha(tablero, bitInicial)` → devuelve el valor de 3 bits (0-7) que hay ahí.
- `escribirFicha(tablero, bitInicial, valor)` → sobrescribe esos 3 bits sin tocar los vecinos.
- `bytesNecesarios(filas, columnas)` → cuántos bytes reservar como mínimo.
- `crearTablero(filas, columnas)` → reserva memoria dinámica inicial.

- `redimensionarTablero(...)` → crea nueva tira, copia bits válidos, libera la vieja (aplica reglas del 65%).

Módulo 2 — visualizacion

- `mostrarTableroBinario(tablero, filas, columnas)` → imprime la tira cruda en bits/bytes (para depurar y para el requisito de “formato binario”).
- `mostrarTableroFichas(tablero, filas, columnas)` → traduce cada posición a un carácter visible (ej: A, B, C... o símbolos) y lo imprime como grilla.

Módulo 3 — juego (reglas)

- `generarFichaAleatoria()` → número 0-5 con distribución uniforme.
- `inicializarTablero(...)` → llena todo el tablero con fichas aleatorias válidas al empezar.
- `eliminarFicha(fila, columna)` → marca esa posición como “vacía” (código 110).
- `aplicarGravedad(...)` → hace caer las fichas sobre los espacios vacíos y rellena arriba con nuevas.
- `detectarCombinacionesHorizontales(...)` / `detectarCombinacionesVerticales(...)` → devuelven qué posiciones deben eliminarse.
- `resolverCombinaciones(...)` → marca esas posiciones como vacías (puede fusionar resultados de horizontal + vertical sin duplicar).
- `procesarCascadas(...)` → repite: detectar → resolver → gravedad → detectar... hasta que no haya más combinaciones. Cuenta cuántas veces se repitió.

Módulo 4 — estructura (agregar/quitar filas y columnas)

- `agregarFila(indice)`, `eliminarFila(indice)`, `agregarColumna(indice)`, `eliminarColumna(indice)`.
- Internamente usan el módulo de memoria para redimensionar y recolocar bits, y después llaman a `procesarCascadas` porque el enunciado exige revisar si se formaron combinaciones nuevas tras el cambio estructural.

Módulo 5 — estado (puntaje y estadísticas)

Variables globales (documentadas) para: dimensiones actuales, eliminaciones del usuario, fichas eliminadas totales, combinaciones detectadas, cascadas de la jugada actual, puntuación. Funciones simples para actualizar y consultar cada una.

main.cpp

Solo orquesta: crea el tablero, muestra menú/interfaz Qt, llama funciones de los otros módulos según la acción del usuario. No debería tener lógica de bits ni de reglas de juego directamente — solo coordina.

Regla de oro: una vez que “Leer ficha” y “Escribir ficha” funcionan perfecto, **ningún otro módulo vuelve a tocar bits directamente**. Todo lo demás (gravedad, combinaciones, redimensionar) trabaja en términos de “fila, columna, valor de ficha” y delega la traducción a bits siempre a esas dos funciones. Esto es lo que evita que el código se vuelva espagueti.

6. Plan de acción — de lo micro a lo macro, en fases

La idea es no intentar todo el juego de una vez. Cada fase debe funcionar de forma aislada antes de pasar a la siguiente.

Fase 1 — Laboratorio de bits (sin juego todavía) Programa mínimo que reserva una tira pequeña de memoria y prueba leerFicha/escribirFicha con casos que fuercen que una ficha quede partida entre dos bytes. Si esto no funciona perfecto, nada más va a funcionar. Prioridad absoluta.

Fase 2 — Tablero visual Reservar memoria para $F \times C$, llenar con valores aleatorios 0-5, e imprimir en pantalla (binario y como fichas). Aquí se prueba crearTablero, generarFichaAleatoria, y ambas funciones de visualización.

Fase 3 — Eliminar + gravedad (sin combinaciones aún) El usuario elige una coordenada, se marca como vacía, y las fichas de arriba caen. Se rellenan los huecos superiores con fichas nuevas. Se imprime el resultado.

Fase 4 — Combinaciones y cascadas Se agrega el detector de 3+ en línea (horizontal y vertical), se resuelven, se llama de nuevo a gravedad, y se repite hasta estabilizar. Se cuenta cuántas cascadas hubo.

Fase 5 — Estructura dinámica (agregar/quitar filas y columnas) La parte más delicada: cambia el tamaño real de la tira de bits, hay que recolocar todo, respetar la regla del 65%, y verificar cascadas después del cambio.

Fase 6 — Interfaz Qt + integración final Una vez que la lógica de consola funciona, se envuelve en la interfaz gráfica de Qt.

7. Cómo dividirse el trabajo (sugerencia, ajústেনla a como trabajan ustedes)

Una vez tengan el análisis claro juntos:

- **Persona A — bajo nivel:** Módulo memoria completo (Fase 1), visualización, y Módulo estructura (Fase 5). Trabaja mucho con `&`, `|`, `<<`, `>>`.
- **Persona B — lógica de juego:** Módulos juego y estado (Fases 3 y 4). Usa siempre leerFicha/escribirFicha de la Persona A, nunca toca bits directamente.

La interfaz Qt (Fase 6) y el `main.cpp` se arman juntos al final, cuando ambas partes ya están probadas por separado.

8. Cosas a no olvidar para el informe preliminar (entrega del 11 de septiembre)

Según el enunciado, esta primera entrega es de **análisis y diseño**, no de código funcionando. Debe incluir: - Contextualización del problema (qué es, qué se pide). - Análisis (como este documento, pero escrito por ustedes). - Diseño: esquema de módulos, funciones, y cómo se relacionan fila/columna → índice → bit → byte.

No hace falta tener código corriendo todavía para esta fecha — hace falta tener la lógica clara en papel, que es exactamente lo que están armando ahora.

9. Preguntas para resolver en pareja antes de escribir código

1. ¿Qué caracteres van a usar para mostrar cada ficha en pantalla? (el enunciado pide elegir algo que se vea bien)
2. ¿Qué código de 3 bits usan para “vacío” y cuál (si acaso) para “especial”?
3. ¿Cómo van a recorrer el tablero para detectar combinaciones sin necesitar estructuras extra tipo vector? (pista: arreglos dinámicos simples de enteros, del mismo estilo que el tablero, o recorridos con contadores)
4. ¿Cuál va a ser su criterio de puntuación? (el enunciado permite definirlo, pero debe quedar documentado)
5. ¿Cómo van a probar que leerFicha/escribirFicha funcionan bien en los tres casos de la Figura 3 (ficha en un solo byte, partida 2+1, partida 1+2)?